Supervised Learning

Support Vector Machines (SVMs)

Hyperplanes

- A hyperplane is an affine subspace of dimension $p-1$:

$$\beta_0 + \beta_1 X_1 + ... + \beta_p X_p = 0.$$

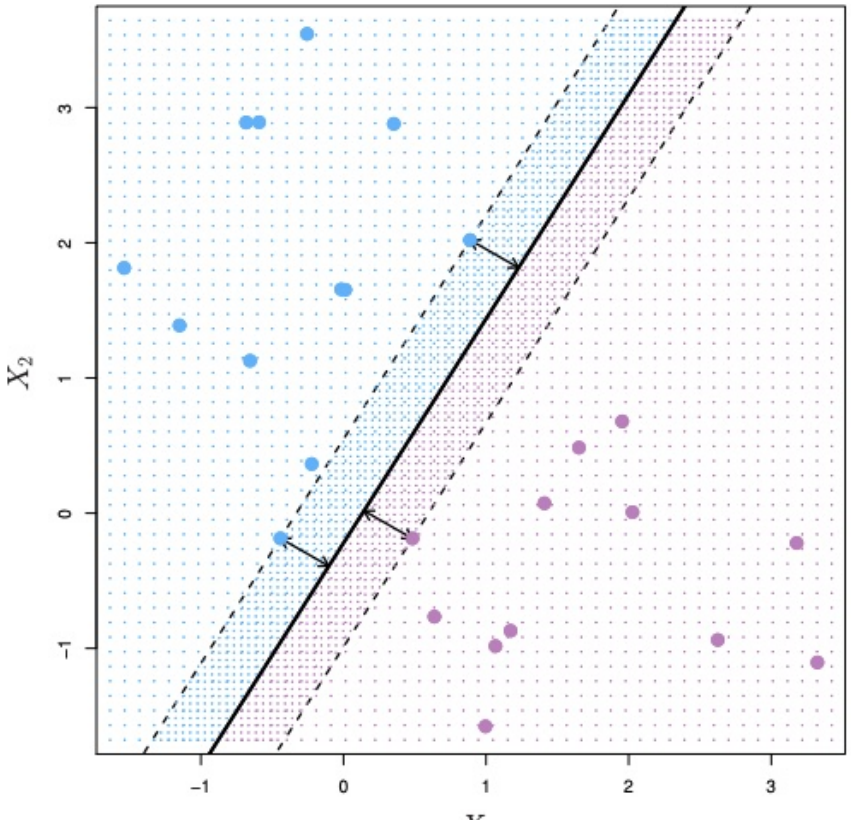
- Halfspaces.

$$\begin{aligned} H^+ &= \{(x_1, ..., x_p) \in \mathbb{R}^p | \beta_0 + \beta_1 x_1 + ... + \beta_p x_p > 0\} \\ H^- &= \{(x_1, ..., x_p) \in \mathbb{R}^p | \beta_0 + \beta_1 x_1 + ... + \beta_p x_p < 0\} \end{aligned}$$

- Example: when $p = 2 \Rightarrow$ a line in $\mathbb{R}^2$.
- $\beta = (\beta_1, ..., \beta_p)^T$: the normal vector (orhogonal to the surface of the hyperplane).

Maximal Margin Classifier

Assume a binary classification problem ($Y \in \{-1, +1\}$). A dataset is well separated if a hyperplane can divide the dataset so that instances of one class ($Y = -1$) fall on one side of the hyperplane, while instances of the other class ($Y = 1$) fall on the other side of the hyperplane.



- the line with the widest margin is the "'best line";
- the points on the margin's edge are the *support vectors*.

Maximal Margin Classifier (Optimization)

Dataset: $\{(\mathbf{x}_i, y_i) \,|\, \mathbf{x}_i = (x_{i1}, .., x_{ip}) \in \mathbb{R}^p,\ y_i \in \{-1, 1\}\}$.

$$\max_{\beta_0, \beta_1, \ldots, \beta_p} \quad M$$

$$\text{such that} \quad \sum_{j=1}^{p} \beta_j^2 = 1$$

$$y_i \left(\beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}\right) \geq M$$

$$i = 1, \ldots, n$$

What if the data is not separable or noisy?

Support Vector Classifier

The Support Vector Classifier (SVC) maximizes a soft margin.

$$\max_{\beta_0,\beta_1,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n} \quad M$$

$$\text{such that} \quad \sum_{j=1}^{p} \beta_j^2 = 1$$

$$y_i \left(\beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}\right) \geq M(1 - \epsilon_i)$$

$$\epsilon_i \geq 0, i = 1, \ldots, n, \sum_{i=1}^{n} \epsilon_i \leq C.$$

Support Vector Classifier

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
from sklearn.datasets import make_blobs
from sklearn.inspection import DecisionBoundaryDisplay
```

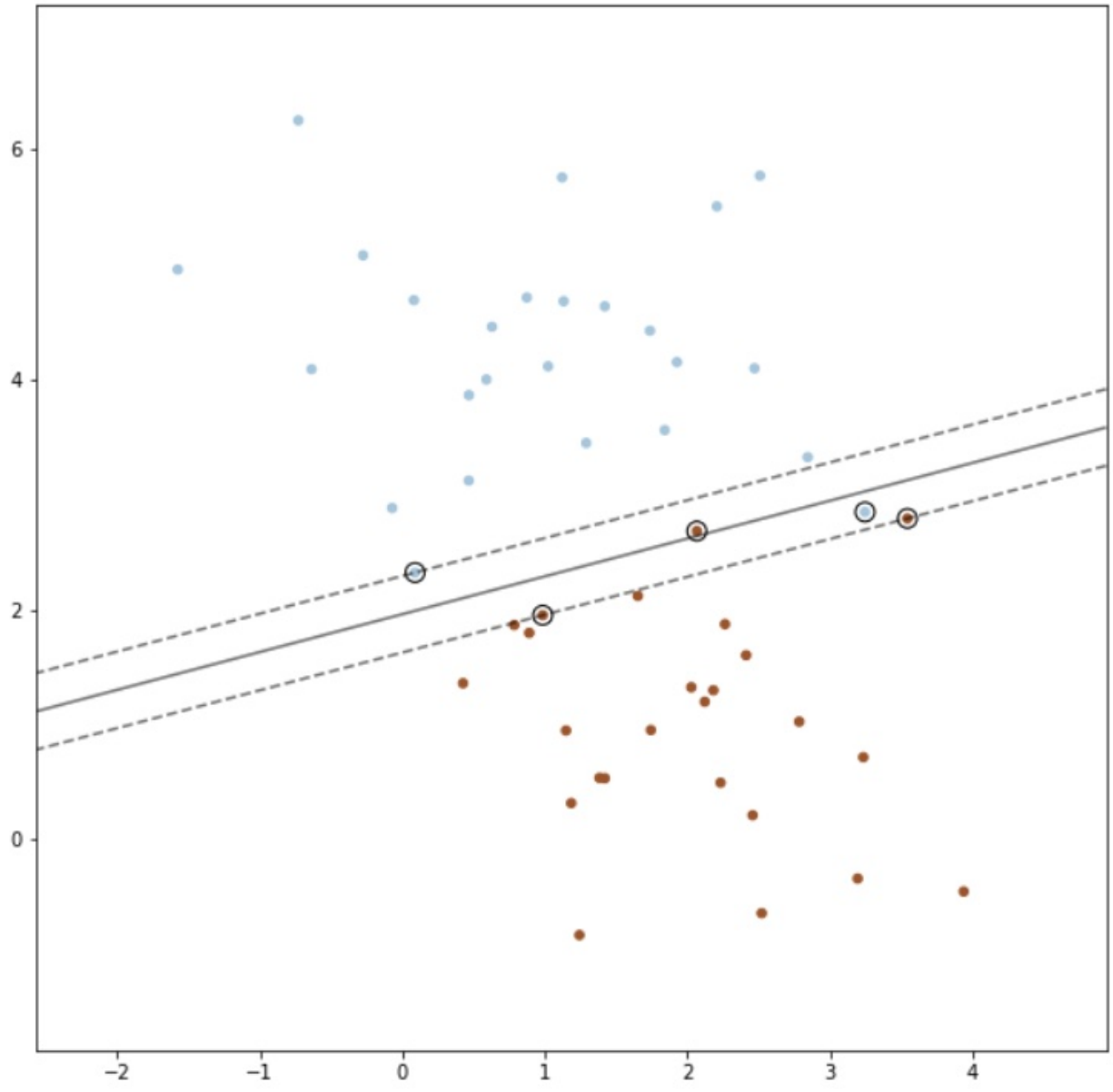
```
# generate sample data
X,y = make_blobs(n_samples=50, centers = 2, random_state=0)
```

```
# fit SVM model
clf = svm.SVC(kernel="linear", C=10)
clf.fit(X,y)
```

```
plt.rcParams["figure.figsize"] = (10,10)
plt.scatter(X[:, 0], X[:, 1], c=y, s=20, cmap=plt.cm.Paired)

# plot the decision function
ax = plt.gca()
DecisionBoundaryDisplay.from_estimator(
    clf,
    X,
    plot_method="contour",
    colors="k",
    levels=[-1, 0, 1],
    alpha=0.5,
    linestyles=["--", "-", "--"],
    ax=ax,
)
# plot support vectors
ax.scatter(
    clf.support_vectors_[:, 0],
    clf.support_vectors_[:, 1],
    s=100,
    linewidth=1,
    facecolors="none",
    edgecolors="k",
)
plt.show()
```

Support Vector Classifier



Support Vector Machines

The Support Vector Machine (SVM) is an extension of SVC that results from enlarging the feature space using *kernels*.

- Let $\mathbf{x}_i, \mathbf{x}_{i'} \in \mathbb{R}^p$. The inner product $\langle \mathbf{x}_i, \mathbf{x}_{i'} \rangle$,

$$\langle \mathbf{x}_i, \mathbf{x}_{i'} \rangle = \sum_{j=1}^{p} x_{ij} x_{i'j}.$$

- The linear support vector classifier can be represented as

$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^{n} \alpha_i \langle \mathbf{x}, \mathbf{x}_i \rangle.$$

It turns out that α_i is non-zero only when $\mathbf{x}_i$ is a support vector

$$\Rightarrow f(\mathbf{x}) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle \mathbf{x}, \mathbf{x}_i \rangle,$$

where $\mathcal{S}$ is the collection of support vector indices.

Support Vector Machines

- The (linear) support vector classifier:

$$f(\mathbf{x}) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle \mathbf{x}, \mathbf{x}_i \rangle,$$

  where $\mathcal{S}$ is the collection of support vector indices.

- If we replace the inner product $\langle \mathbf{x}_i, \mathbf{x}_{i'} \rangle$ by a kernel, $K(\mathbf{x}_i, \mathbf{x}_{i'})$, $\Rightarrow$ the SVM

$$f(\mathbf{x}) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(\mathbf{x}, \mathbf{x}_i)$$

- Nonlinear kernels:
  - *Polynomial kernel* $K(\mathbf{x}_i, \mathbf{x}_{i'}) = \left(1 + \sum_{j=1}^{p} x_{ij} x_{i'j}\right)^d$;
  - *Radial kernel* $K(\mathbf{x}_i, \mathbf{x}_{i'}) = exp\left(-\gamma \sum_{j=1}^{p} \left(x_{ij} - x_{i'j}\right)^2\right)$, where $\gamma > 0$.